LINUX FOUNDATIONS

Linux Foundations

Operate the system. Trust the evidence.

Terminal · permissions · SSH · logs · systemd

> Full route · 6.5 hours
> 3 h theory + demos · 3.5 h labs

Ubuntu 24.04 browser labs · macOS ✦ Windows

By the end, you can recover your bearings

01

Navigate

Move through Linux, manipulate files, and use pipes without losing track of location.

02

Control access

Reason about users, groups, ownership, modes, sudo, and SSH keys.

03

Diagnose

Start and stop services, filter logs, identify a cause, and verify the repair.

ENVIRONMENT

Your browser is a window into another Linux system

BROWSER

Safari, Edge, or Chrome on macOS or Windows

Shows instructions + terminal

`http://linux-lab.local`

SESSION

Ubuntu 24.04 in an Incus container

Runs ls, sudo, apt, systemctl

`learner@linux-lab:~$`

Browser outside. Linux inside.

Ten theory → demo → lab cycles

180 min

Theory ✦ distinct demos

- Linux boundary + filesystems
- Shell evidence + software
- Identity + SSH + systemd

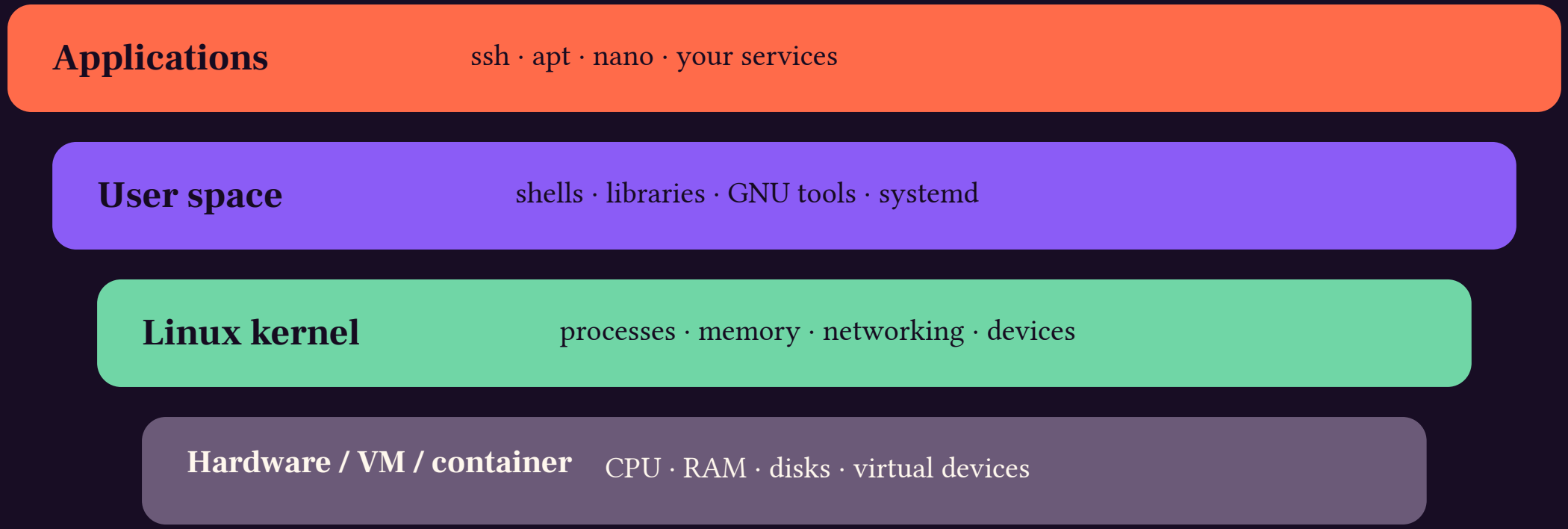
210 min

Ten required labs

- Open an isolated incident
- Predict, solve, verify state
- Processes + drop-ins included

Full route: 390 instructional minutes. The six-hour variant moves 30 minutes outside the room.

Linux is the kernel; a usable system is a stack



The kernel turns hardware into abstractions

Processes	scheduled programs with isolated address spaces
Memory	virtual memory, pages, caches, and protection
Filesystems	one namespace across disks and virtual data
Network	sockets, routes, interfaces, and packet flow
Devices	drivers expose hardware through common interfaces

A distribution is a curated operating agreement

Same fundamentals

- Kernel interface
- Filesystem concepts
- Users and permissions
- OpenSSH + systemd

Different choices

- Package format
- Release cadence
- Default configuration
- Support lifecycle

Choose for context

- Ubuntu / Debian
- RHEL / Rocky / Alma
- Fedora
- Cloud/vendor images

Today: Ubuntu 24.04 LTS keeps every lab consistent.

Distribution families share ideas, not every command

Debian family

- Ubuntu · Debian
- APT + .deb
- /etc/apt
- Long support

Red Hat family

- RHEL · Rocky · Alma
- DNF + RPM
- /etc/yum.repos.d
- SELinux defaults

Other contexts

- Fedora: newer stacks
- Alpine: small images
- Arch: rolling
- Vendor clouds

Identify the family before copying package or service instructions.

Stable and rolling releases trade currency

STABLE / LTS

predictability

- Longer support window
- Security fixes with fewer surprises
- Older major versions are common

ROLLING

currency

- Frequent package updates
- Fast access to new features
- More integration change over time

Choose for workload and support needs, not fashion.

Virtual machines isolate kernels; containers share one

VIRTUAL MACHINE

`hardware boundary`

- Guest owns a kernel
- Different isolation boundary
- Higher memory and boot cost

SYSTEM CONTAINER

`OS boundary`

- Shares the host kernel
- Own user space and systemd
- Fast cloning and reset

The browser labs use Incus system containers inside a controller VM.

Live demo: locate host, VM, and container facts

Host operating system	`sw_vers or systeminfo`
Controller kernel	`uname -srmo`
Instance type	`incus info demo-seat`
Container user space	`cat /etc/os-release`
Container PID 1	`ps -p 1 -o comm=`
Boundary claim	`shared kernel; isolated user space`

Lab 1 — Profile the system before changing it

MISSION · 15 MIN

System orientation

DELIVERABLES

```
Create ~/system-profile.txt
Create ~/path-map.txt
Derive every value from this seat
```

ACCEPTANCE EVIDENCE

Checker requires

- Ubuntu, kernel, and architecture
- PID 1 is systemd
- /etc, /var/log, and /run exist
- No guessed values

Debrief: which fact describes user space, and which describes the kernel?

Core operating questions transfer across distributions

Locate

- Where am I?
- Which host?
- Which user?
- Which file or unit?

Change

- Predict one state
- Use narrow privilege
- Make one edit
- Capture exit status

Verify

- Inspect resulting state
- Read relevant logs
- Test intended identity
- Prove behavior

Tools differ; the evidence loop remains stable.

Most investigations begin at five filesystem anchors

`/`	one root; every absolute path starts here
`/etc`	host-specific configuration
`/home`	human users' files
`/var`	changing data: logs, caches, queues
`/usr`	installed programs, libraries, shared data

Filesystem paths express intent, not physical disks

`/`	the root of the visible namespace
`Mount`	attach another filesystem at a directory
`Path`	a route through directory entries
`File`	a name referring to an inode-like object
`Metadata`	owner, mode, timestamps, size, and type

The hierarchy separates configuration, data, and runtime state

Configuration

- /etc host settings
- /usr shipped defaults
- Prefer drop-ins
- Track intentional edits

Persistent data

- /home user files
- /var changing service data
- /srv served content
- /opt optional software

Runtime state

- /run since boot
- /tmp temporary files
- /proc process views
- /sys device and kernel views

Directory purpose helps you search before you know an exact filename.

/proc and /sys expose live kernel state

/proc

process + kernel state

- PID directories describe processes
- cpuinfo and meminfo expose summaries
- Values may change between reads

/sys

devices + kernel objects

- Hardware and driver relationships
- Many tunables are writable by root
- Changes can affect the running system

Read freely; write only with a documented reason.

Paths are coordinates; your working directory is the origin

ABSOLUTE

STARTS AT /

```
/var/log/syslog
/etc/ssh/sshd_config
```

Works from anywhere because it begins at the root.

RELATIVE

STARTS AT pwd

```
../archive/report.txt
./scripts/check.sh
```

Meaning changes when your working directory changes.

Safety check: pwd → inspect the target → change it → verify the result

Path syntax has a small, precise vocabulary

/name	absolute: begin at root
name	relative: begin at the working directory
.	the current directory
..	the parent directory
~	shell expands to user's home
name/	directory path; trailing slash clarifies intent

Hidden names are a convention, not protection

NORMAL LISTING

`ls`

- Omits names beginning with dot
- Keeps routine output quieter
- Permissions are unchanged

ALL ENTRIES

`ls -la`

- Shows dotfiles and metadata
- Useful for .ssh and configuration
- Still governed by directory access

A dot changes default visibility; permissions control access.

The first character of ls -l identifies object type

-	regular file
d	directory
l	symbolic link
c / b	character or block device
s	Unix-domain socket
p	named pipe

Hard links and symbolic links fail in different ways

HARD LINK

another name

- Points to the same inode
- Usually cannot cross filesystems
- Data remains until last link is removed

SYMBOLIC LINK

stored path

- Can cross filesystems
- Can point to a directory
- Can become dangling

Inspect with ls -li and readlink before changing links.

Globs expand before the command starts

`*`	zero or more characters except leading dot
`?`	exactly one character
`[ab]`	one character from a set
`[0-9]`	one character in a range
`Quoted glob`	literal pattern; no pathname expansion

Inspect targets before changing files

1	2	3	4	5
Locate	**Enumerate**	**Predict**	**Change**	**Verify**
pwd and realpath	printf or find the exact targets	state the expected names and count	cp, mv, or rm with narrow scope	list, compare, or test the result

Interactive flags help, but a verified target list is stronger.

Live demo: diagnose a wrong directory

LOCATE → FAIL SAFELY → RESOLVE → VERIFY

```
cd /var/tmp/demo-tree/archive
pwd; ls -la
cat report.txt; printf 'status=%s\n' "$?"
realpath ../incoming/report.txt
namei -l ../incoming/report.txt
cp ../incoming/report.txt ./report.reviewed
stat -c '%F %a %U:%G %n' ./report.reviewed
```

The failed relative path is evidence about coordinates, not permissions.

Lab 2 — Prove how filesystem names behave

MISSION · 25 MIN

Filesystem and links

DELIVERABLES

```
Organize ~/projects/linux-course
Copy welcome; archive the draft
Create hard and relative symbolic links
```

ACCEPTANCE EVIDENCE

Checker requires

- Required directory tree exists
- Copy remains; moved source is absent
- Hard link shares an inode
- Symlink stores ../notes/welcome.txt

Debrief: which names refer to one object, and which name stores a path?

The shell turns text into a process request

PROMPT ✦ COMMAND

```
ubuntu@linux-course:~/demo$ grep -i error app.log
```

grep	-i	error	app.log
program	option	argument	argument

Exit status 0 = success · non-zero = a condition or failure

Terminal, shell, and command are separate layers

TERMINAL

`input + display`

- Keyboard and screen interface
- Local app or browser ttyd
- Transports control characters

SHELL

`parser + launcher`

- Expands variables and globs
- Connects pipelines and redirections
- Starts programs and reports status

The command is the program plus the arguments produced by the shell.

Quoting decides what the shell expands

`Unquoted`	split words, expand variables, expand globs
`'single quotes'`	preserve every character literally
`"double quotes"`	expand variables but preserve one argument
`backslash`	escape the next character
`$()`	replace with captured command output

Shell expansion explains surprising commands

Parse	recognize quotes, operators, and command boundaries
Parameters	replace $name and ${name}
Commands	replace $(...) with output
Split	unquoted results may become several words
Glob	patterns may become many pathnames
Execute	run the final program and arguments

Exit status reports success or failure

`0`	the command reports success
`1–125`	program-defined condition or failure
`126`	found but not executable
`127`	command not found
`128 + signal`	terminated by a signal

Control operators and pipes solve different jobs

&& AND ||

`control flow`

- Run next only on success
- Or run fallback on failure
- Status determines the branch

| PIPE

`data flow`

- Connect stdout to stdin
- Commands run as a pipeline
- Last status is reported by default

Use control operators for decisions and pipes for streams.

Navigate by asking, moving, then confirming

Where am I?	`pwd`
What is here?	`ls -la`
Move	`cd /var/log`
Create	`mkdir -p notes/archive`
Inspect	`cat · less · head · tail`
Find	`find · grep`

Use local help before trusting a random command

Quick syntax

- command --help
- Short option summary
- Examples may be included
- Best for recall

Full manual

- man command
- /pattern to search
- n for next result
- q to leave

Shell built-ins

- help cd
- type command
- command -V name
- distinguish aliases

Local documentation matches the installed version.

Pipes connect small tools into an investigation

ONE QUESTION, THREE FILTERS

```
journalctl -u ssh --since today | grep -i failed | tail -n 20
```

| send output to the next command

> replace a file with output

>> append output to a file

Rule: inspect before ◂ — redirection happens before the command runs.

Processes start with three standard streams

`stdin`	file descriptor 0: keyboard or pipe input
`stdout`	file descriptor 1: normal output
`stderr`	file descriptor 2: diagnostics
`>`	replace a file with stdout
`2>`	replace a file with stderr

Build pipelines as readable evidence stages

1	2	3	4	5
Question	**Source**	**Filter**	**Transform**	**Verify**
state the answer you need	choose the smallest relevant input	remove unrelated records	extract, sort, or count	inspect samples and totals

A long pipeline is maintainable when each stage has one job and can be tested independently.

Live demo: derive a pipeline from record structure

INSPECT FORMAT

```
head auth.log
```

- Confirm timestamp + key=value fields
- Ask: which sources failed?
- Keep diagnostics visible

TRANSFORM DELIBERATELY

```
filter → extract → sort → count
```

- Save selected records first
- Count the saved evidence
- Explain each stage aloud

Lab 3 uses different files and record grammars; copy-paste will not solve it.

Nano is enough to make a safe first edit

NANO

```
nano ~/linux-course/notes/day1.txt

^O  Write Out / save
^X  Exit
^W  Search
^K  Cut line
^U  Paste line
```

^ means Ctrl

- Open the exact path
- Make one small change
- Save, exit, then verify
- Use sudoedit for protected files

Editing is not verification.

Safe edits use backup, diff, and validation

1

Inspect

read the current file and ownership

2

Protect

copy or use version control where appropriate

3

Edit

nano or sudoedit the exact path

4

Validate

use the program's syntax checker

5

Apply

reload or restart, then verify behavior

Saving a file proves only that bytes changed. It does not prove the configuration is valid or active.

Lab 3 — Turn configuration and logs into evidence

MISSION · 20 MIN

Editor and text evidence

DELIVERABLES

```
Edit app.conf safely in Nano
Save normalized HTTP status counts
Save exactly two ERROR records
```

ACCEPTANCE EVIDENCE

Checker requires

- Three production settings, no duplicates
- STATUS COUNT order is preserved
- 404 appears exactly three times
- errors.txt contains two records

Debrief: where did record structure determine the pipeline?

APT separates repositories, indexes, and installed files

Repository

- Signed metadata
- Package archives
- Release channels
- Repository trust keys

Local index

- Refreshed by apt update
- Available versions
- Candidate selection
- Dependency metadata

Installed state

- Tracked by dpkg
- Files on disk
- Package status
- APT history log

apt update refreshes knowledge; apt install changes installed state.

Use APT with an inspect-change-verify rhythm

Refresh	`sudo apt update`
Search	`apt search tree`
Inspect	`apt show tree`
Install	`sudo apt install tree`
Verify	`dpkg -s tree; command -v tree`
Audit	`less /var/log/apt/history.log`

Live demo: connect jq to repository and file ownership

CANDIDATE → INSTALL → REGISTER → OWN → RUN

```
apt-cache policy jq
sudo apt-get update
sudo apt-get install -y jq
dpkg-query -W -f='${Status}\n' jq
command -v jq
dpkg-query -S "$(command -v jq)"
printf '{"state":"ready"}\n' | jq -r .state
```

Repository state, package state, file ownership, and behavior answer different questions.

Lab 4 — Verify a package through five views

MISSION · 20 MIN

Package lifecycle

DELIVERABLES

```
Refresh repository metadata
Install tree through APT
Write package and behavior reports
```

ACCEPTANCE EVIDENCE

Checker requires

- Candidate inspected before install
- dpkg says install ok installed
- /usr/bin/tree belongs to tree
- tree output proves behavior

Debrief: which command proves repository state, installed state, and behavior?

Linux authorizes identities, not job titles

USER

UID 1001
alice

PRIMARY ✦ SUPPLEMENTARY GROUPS

alice · courseops · sudo

IDENTITY CHECKS

```
whoami
id
groups
getent passwd alice
getent group courseops
```

Processes run as a user.
Files have an owner ✦ group.
Access checks combine both.

Names are convenient; numeric IDs drive authorization

Username	human-readable account label
UID	numeric user identity used by the kernel
Group name	human-readable collaboration label
GID	numeric group identity stored on files
Effective IDs	identity currently used for access checks

Account databases separate public identity from secrets

/etc/passwd

- name and UID
- primary GID
- home directory
- login shell

/etc/group

- group name and GID
- supplementary members
- not every primary member
- readable account data

/etc/shadow

- password hashes
- aging fields
- root-readable only
- never copy casually

Use getent so local files and network identity sources are queried consistently.

Existing shells keep old group credentials

ACCOUNT DATABASE

updated now

- usermod changes membership
- getent shows the new record
- Future sessions inherit it

CURRENT SHELL

old credentials

- Keeps groups from login time
- id may not show the change
- Re-login or newgrp refreshes context

Always test membership in the process that will do the work.

Use sudo for one deliberate action

Normal shell

least privilege
clear ownership
smaller blast radius

✦ sudo

One privileged command

EXPLICIT INTENT

```
sudo apt install tree
sudo systemctl restart ssh
sudoedit /etc/ssh/sshd_config
```

Avoid living in a root shell. Ask: what exact command needs privilege?

sudo policy answers who may run what as whom

Caller	the authenticated invoking user
Target	root by default or selected with -u
Command	the permitted executable and arguments
Policy	rules from sudoers and included files
Audit	logs record the privileged request

Permissions are three triplets evaluated in order

```
- rwx r-x ---
```

OWNER	GROUP	OTHERS
rwx	r-x	---

file: read · change · run directory: list names · create/remove · enter/traverse

Directory r, w, and x control different abilities

Read · r

- list entry names
- does not reveal metadata alone
- ls may show names only
- needs x for useful lookup

Write · w

- create entries
- remove entries
- rename entries
- usually requires x too

Execute · x

- traverse the directory
- look up named entries
- reach deeper paths
- not ‘run the directory’

For directories, x means traversal and w changes names in that directory.

Parent directories participate in every path lookup

`/`	traverse root
`/srv`	traverse service data
`/srv/team`	traverse team directory
`file`	apply file read or write bits
`namei -l`	display every component and its mode

4 ✦ 2 ✦ 1 makes numeric modes readable

`r = 4` `w = 2` `x = 1`

---

640 = `rw-  r--  ---`

SAME INTENT, VERIFIED

```
chmod 640 report.txt
chmod u=rw,g=r,o= report.txt
stat -c "%A %a %U:%G %n" report.txt
```

Symbolic chmod states exactly who and what changes

u	owner class
g	group class
o	other class
✦ / -	add or remove selected bits
=	set the selected class exactly
chmod g✦r	add group read without changing other bits

umask removes permissions; it never grants

REQUESTED MODE

`program default`

- Files commonly request 666
- Directories commonly request 777
- Execute is not added to ordinary files

UMASK

`bits to clear`

- 022 yields files 644
- 022 yields directories 755
- Process-local and inherited

Final mode = requested permissions with masked bits removed.

setgid keeps shared directories in one group

`Directory owner`	usually an administrator or service account
`Directory group`	the collaborating team
`Mode 2xxx`	setgid bit on the directory
`New entries`	inherit the directory group
`Verification`	stat and create a file as a real member

Sticky protects names in shared directories

WITHOUT STICKY

`shared write`

- Any writer may remove entries
- File ownership does not protect names
- Easy for users to disrupt each other

WITH STICKY

`mode 1xxx`

- Only owner, directory owner, or root removes
- Typical example is /tmp
- Shown as t in the other execute position

Sticky controls deletion and rename, not file confidentiality.

ACLs extend permissions without replacing the base model

MODE BITS

three classes

- Owner, group, other
- Visible in ls -l
- Portable baseline

POSIX ACL

named entries

- Additional users or groups
- Mask limits effective named rights
- Inspect with getfacl

Use ACLs deliberately and document why the exception exists.

Live demo: repair /srv/demo-share for demoops

IDENTITY → PATH → MODE → REAL OPERATION

```
id demo-alex; id demo-riley
namei -l /srv/demo-share/brief.txt
sudo -u demo-alex test -w /srv/demo-share || echo BLOCKED
sudo chown root:demoops /srv/demo-share /srv/demo-share/brief.txt
sudo chmod 2770 /srv/demo-share
sudo chmod 0660 /srv/demo-share/brief.txt
sudo -u demo-riley sh -c 'echo riley > /srv/demo-share/riley.txt'
stat -c '%A %a %U:%G %n' /srv/demo-share/*
```

Test policy with the identities that must succeed and fail.

Lab 5 — Repair collaboration without broad access

MISSION · 30 MIN

Permission incident

DELIVERABLES

```
Repair /srv/team-share for webteam
Preserve a safe existing-file mode
Test Alice, Bob, and outsider
```

ACCEPTANCE EVIDENCE

Checker requires

- Directory is root:webteam, mode 2770
- checklist.txt is group-writable 660
- New files inherit webteam
- Outsider remains blocked

Debrief: which directory permission made creation possible, and why setgid?

SSH authenticates both ends of the conversation

CLIENT

“Is this really my server?”

Checks the server host key
against known_hosts

SERVER

“Is this user authorized?”

Checks a signature against
authorized_keys

Encryption without host verification can still connect you to an attacker.

SSH separates transport, host, and user trust

`Transport`	encrypts and protects session integrity
`Host key`	proves which server endpoint answered
`User proof`	proves which account may log in
`Account policy`	server maps proof to account policy
`Session`	starts command, shell, forwarding, or subsystem

Host keys identify servers; user keys identify people

HOST KEY

server identity

- Private half stays on server
- Client stores public fingerprint
- Change requires investigation

USER KEY

user identity

- Private half stays with user
- Server stores public key
- Can be revoked per account

Never copy either private key to the opposite side.

Verify host keys through an independent channel

1

Obtain

get the expected fingerprint from an administrator

2

Connect

observe the presented fingerprint

3

Compare

match algorithm and complete fingerprint

4

Accept

store the verified key in known_hosts

5

Recheck

treat later changes as an event

Typing yes without comparison is trust on first use, not verification.

The private key proves; the public key permits

PRIVATE KEY

`~/.ssh/id_ed25519`

- Stays on the client
- Protect with a passphrase
- Never email or copy to server

PUBLIC KEY

`~/.ssh/id_ed25519.pub`

- May be copied to servers
- Stored in authorized_keys
- Can be revoked independently

GENERATE ON THE CLIENT

```
ssh-keygen -t ed25519 -a 64 -C "linux-course"
```

A passphrase and agent reduce private-key exposure

Phrase	a passphrase encrypts the private key file at rest
KDF rounds	increase cost of password guessing
Agent	holds an unlocked key for a limited session
Permissions	prevent other local users from reading the file
Rotation	replace compromised or obsolete credentials

OpenSSH checks every component of the authorization path

Home directory	`owned by the account; not broadly writable`
~/.ssh	`normally mode 700`
authorized_keys	`normally mode 600`
Public key line	`complete algorithm and key data`
Account	`login shell and policy must allow access`
Server log	`explains rejected ownership or mode`

Put safe SSH defaults in a named client profile

~/.ssh/config

```
Host linux-course
    HostName 192.168.64.10
    User trainee
    IdentityFile ~/.ssh/linux_course_ed25519
    IdentitiesOnly yes
    StrictHostKeyChecking accept-new
    ServerAliveInterval 60
```

- One alias replaces repeated flags
- IdentitiesOnly avoids key roulette
- Verify the first host fingerprint
- Use “yes” after onboarding for strictness

Same OpenSSH syntax on macOS and Windows.

SSH uses the first value obtained

`Command line`	explicit options for this invocation
`User config`	~/.ssh/config patterns in order
`System config`	/etc/ssh/ssh_config defaults
`Host patterns`	specific entries should appear before broad ones
`Effective view`	ssh -G alias prints resolved settings

Live demo: verify a separate SSH endpoint on port 2207

SERVER FINGERPRINT → USER AUTHENTICATION

```
ssh-keyscan -t ed25519 -p 2207 localhost > host.scan
ssh-keygen -lf /run/demo-sshd/ssh_host_ed25519_key.pub -E sha256
ssh-keygen -lf host.scan -E sha256
install -m 600 host.scan demo-known_hosts
ssh -o BatchMode=yes -o StrictHostKeyChecking=yes \
  -o UserKnownHostsFile=demo-known_hosts \
 -i demo-client/id_ed25519 -p 2207 \
  demo-operator@localhost 'id; hostname'
```

Host trust and user trust are separate claims with separate keys.

Test SSH failures one layer at a time

1	2	3	4	5
Network	**Host trust**	**Client choice**	**User auth**	**Server policy**
resolve name and reach the port	inspect known_hosts and fingerprint	ssh -G shows user, key, and port	ssh -vv reveals offered and accepted methods	read ssh service logs and account state

Verbose output is most useful after the basic target, port, and identity are confirmed.

Lab 6 — Establish host and user trust with SSH

MISSION · 25 MIN

SSH trust setup

DELIVERABLES

```
Generate a dedicated Ed25519 key
Verify the server fingerprint
Build the training-server profile
```

ACCEPTANCE EVIDENCE

Checker requires

- Private material has restrictive modes
- known_hosts is verified before login
- Effective client options are pinned
- BatchMode key-only login succeeds

Debrief: which key authenticates the server, and which key authenticates you?

Linux logs live in files and in the systemd journal

TEXT FILES

`/var/log`

- Application-specific files
- APT and dpkg history
- Rotation changes filenames

STRUCTURED JOURNAL

`journalctl`

- Unit, boot, PID, priority fields
- Time-aware filtering
- May be volatile or persistent

Start with the service and time window—not “all logs”.

A useful log record answers five questions

When	timestamp with timezone or boot context
Where	host, container, or node
Who	unit, process, PID, or identity
What	event and severity
Why next	correlation field, exit code, or causal detail

Filter the journal toward one answer

Which service?	`journalctl -u ssh`
Which time?	`--since "-10 min"`
Which severity?	`-p warning`
How much?	`-n 50 --no-pager`
Live?	`-f`

journalctl can narrow by unit, boot, time, and priority

Unit	`-u ssh.service`
Current boot	`-b`
Previous boot	`-b -1`
Time	`--since '-10 minutes'`
Priority	`-p warning`
Stable output	`-n 50 --no-pager -o short-iso`

Filter narrowly, then widen one dimension at a time

Bound

- Choose one unit
- Select one boot
- Set the time window
- Name the incident

Read

- Follow timestamps
- Find the first cause
- Capture exit reason
- Note dependencies

Widen

- Extend time
- Add priorities
- Inspect related units
- Inspect the process

Do not begin with all logs and search by intuition.

Live demo: separate STALE_CACHE from failed state

UNIT → BOOT → CAUSAL EVENT → RESULT

```
systemctl show course-demo-noisy.service \
  -p ActiveState -p Result -p ExecMainStatus
journalctl -u course-demo-noisy.service -b \
  -o short-iso --no-pager
journalctl -u course-demo-noisy.service -b --no-pager \
  | grep -E 'incident=|code=exited'
```

The application event carrying exit code 17 precedes systemd's consequence.

Lab 7 — Find the causal event in the journal

MISSION · 20 MIN

Journal investigation

DELIVERABLES

```
Preserve unit-scoped journal evidence
Record state, result, incident, and
code
Explain cause versus consequence
```

ACCEPTANCE EVIDENCE

Checker requires

- Unit and current boot are scoped
- ActiveState and Result are distinct
- DISK_THRESHOLD is identified
- Exit code 42 is preserved

Debrief: which line is the application cause, and which line is systemd's result?

systemd manages declared units toward a state

UNIT FILE

What should run?

dependencies · user · command
restart policy · boot target

→

MANAGER

What is running?

start · stop · supervise
track exit status

→

JOURNAL

What happened?

events + output

“Enabled” configures future activation. “Active” describes runtime state. They are independent.

Service transitions leave journal evidence

Declared

- ExecStart command
- Service user
- Dependencies
- Restart policy

Transition

- Conditions
- Ordering
- Credentials
- Exit status

Observed

- Active state
- Substate
- Main PID
- Journal events

systemctl status is a current summary; journalctl provides the longer event sequence.

Requirement and ordering are separate relationships

`Requires=`	start together; failure relationship
`Wants=`	weaker activation relationship
`After=`	ordering only; does not pull a unit in
`Before=`	inverse ordering relationship
`PartOf=`	propagate selected lifecycle actions

Use systemctl verbs deliberately

Observe	`status · is-active · is-enabled`
Change runtime	`start · stop · restart · reload`
Change boot	`enable · disable`
Inspect config	`cat · show`
After unit edits	`daemon-reload`

Restart a service only after capturing evidence.

Live demo: diagnose a missing EnvironmentFile

FAILED DECLARATION → VERIFIED ENDPOINT

```
systemctl status demo-api.service --no-pager
journalctl -u demo-api.service -b --no-pager
systemctl cat demo-api.service
printf 'DEMO_PORT=8181\n' | sudo tee /etc/demo-api.env
sudo systemctl restart demo-api.service
systemctl is-active demo-api.service
ss -ltn 'sport = :8181'
curl -I http://127.0.0.1:8181/
```

Restore the declared prerequisite; then test state, socket, and protocol.

Diagnose services from symptom to verified behavior

1	2	3	4	5
State	**Events**	**Definition**	**Repair**	**Behavior**
systemctl status and is-active	journalctl by unit, boot, and time	systemctl cat and show	change one evidenced cause	test the endpoint as a real client

A green active state proves that systemd sees a running process, not that users receive the intended service.

Read the repair from declaration to behavior

State	`systemctl status demo-api --no-pager`
Events	`journalctl -u demo-api -b`
Definition	`systemctl cat demo-api`
Repair	`create /etc/demo-api.env only`
Restart	`systemctl restart demo-api`
Verify	`is-active + ss :8181 + curl -I`

Lab 8 — Repair a service from evidence

MISSION · 25 MIN

Broken service capstone

DELIVERABLES

```
Explain the CHDIR failure
Create only the missing prerequisites
Enable, start, and verify course-web
```

ACCEPTANCE EVIDENCE

Checker requires

- Web root ownership and mode are correct
- Unit is active and enabled
- Port 8088 is listening
- HTTP returns the health sentence

Debrief: why was creating the directory safer than editing the unit?

A process is a running program with context

PID	a temporary process identifier
Parent	the process that created it
Identity	effective user and groups used for access checks
Environment	name/value data inherited at process start
Descriptors	open files, sockets, and streams

A service is observable through independent interfaces

MANAGER ✦ PROCESS

`unit → MainPID`

- systemctl show exposes runtime state
- ps shows hierarchy and credentials
- /proc/PID/cmdline supports the account

SOCKET ✦ BEHAVIOR

`listener → client`

- ss proves a listening transport endpoint
- curl proves application behavior
- stop through the manager

No single view proves declaration, process, network, and behavior.

Live demo: trace demo-echo.service to port 9191

UNIT → PID → COMMAND → SOCKET → CLIENT

```
systemctl show demo-echo.service -p MainPID -p ExecStart
pid="$(systemctl show demo-echo.service -p MainPID --value)"
ps -o pid,ppid,user,stat,cmd -p "$pid"
sudo tr '\0' ' ' <"/proc/$pid/cmdline"; echo
sudo ss -ltnp 'sport = :9191'
curl -fsS http://127.0.0.1:9191/
sudo systemctl stop demo-echo.service
ss -ltnH 'sport = :9191'
```

Stop a managed process through systemd, then verify both manager and socket state.

Lab 9 — Trace one service through four layers

MISSION · 15 MIN

Process and port investigation

DELIVERABLES

```
Record unit, PID, user, port, and HTTP
Stop the process through systemd
Prove the listener disappears
```

ACCEPTANCE EVIDENCE

Checker requires

- MainPID matches the saved report
- Process identity is courseprobe
- Port 9099 serves the expected body
- Unit inactive and socket closed

Debrief: what does each of systemctl, ps, ss, and curl prove?

Unit files come from layered locations

Vendor units	vendor path varies by distribution
Admin units	/etc/systemd/system and overrides
Runtime units	/run/systemd/system
Drop-ins	name.service.d/*.conf overrides selected settings
Effective view	systemctl cat name.service

Enabled, active, reloaded, and restarted are independent facts

BOOT ✦ RUNTIME

`enable / start`

- enable changes boot links
- start changes current state
- --now requests both operations

CONFIG APPLY

`reload / restart`

- reload asks process to reread
- restart replaces runtime process
- daemon-reload rereads unit files

Name the state you intend to change before choosing the verb.

Live demo: override demo-banner without editing its unit

FRAGMENT → DROP-IN → RELOAD → RESTART → BEHAVIOR

```
systemctl show demo-banner.service \
  -p FragmentPath -p DropInPaths -p Environment
sudo install -d /etc/systemd/system/demo-banner.service.d
printf '[Service]\nEnvironment=DEMO_LOG_LEVEL=debug\n' \
  | sudo tee /etc/systemd/system/demo-banner.service.d/override.conf
sudo systemctl daemon-reload
sudo systemctl restart demo-banner.service
systemctl cat demo-banner.service
cat /run/demo-banner.txt
```

Effective declaration, manager state, and generated behavior must agree.

Lab 10 — Apply a durable systemd override

MISSION · 15 MIN

systemd drop-in

DELIVERABLES

```
Add one administrator drop-in
Reload definitions and restart
Verify declaration and behavior
```

ACCEPTANCE EVIDENCE

Checker requires

- Original fragment remains intact
- DropInPaths includes override.conf
- Environment reports production
- /run/course-mode.txt says production

Debrief: why are daemon-reload, restart, and behavior checks separate steps?

A safe operator closes every change with evidence

1	2	3	4	5
Locate	**Predict**	**Act**	**Observe**	**Verify**
browser or session · user · directory	what state should change?	one scoped command	output · status · logs	behavior through another check

Pair prompt: explain this loop using the failed-service demo.

When blocked, explain the evidence chain

Use this five-step peer check:

- Which user, host, and working directory?
- What target state did you inspect?
- Can you repeat the exact symptom?
- Which log or exit status identifies the cause?
- What one change did you retest?

A green checker starts the explanation; it does not replace it.

Primary manuals, claim ledger, and open-course attribution

OPEN COURSE INSPIRATION

Linux Upskill Challenge · CC BY 4.0
linuxupskillchallenge.com

Linux Training + Linux101Labs
coverage and exercise-pattern review

NORMATIVE REFERENCES

Linux kernel + GNU manuals
FHS 3.0 + Ubuntu Server docs
OpenSSH + systemd manuals
Incus instance documentation

Claim-level traceability resources/CLAIM_LEDGER.md · labs/FACT_CHECK.md

Full URLs, scope qualifications, and licenses: resources/SOURCES.md